

Technologies Utilisees

Djezzy Supervision - PFE 2025/2026

Stack Technique : React, Django, PostgreSQL, Gemini AI

Architecture : Client-Server REST API

Sommaire

1. Architecture Generale
2. Backend : Django REST Framework
3. Base de Donnees : PostgreSQL
4. Frontend : React.js
5. Intelligence Artificielle : Google Gemini
6. Authentification : JWT (Simple JWT)
7. Cartographie : Leaflet.js
8. Bibliothèques Supplémentaires
9. Diagramme des Echanges
10. Choix Techniques et Justifications

1. Architecture Generale

L'application suit une architecture client-server classique avec une separation claire entre le frontend (React) et le backend (Django). Les deux communiquent via une API REST au format JSON, securisee par des tokens JWT.

Schema de l'architecture

React (Frontend) <--> Django REST API (Backend) <--> PostgreSQL (Base de donnees)

Le frontend est une application Single Page Application (SPA) qui communique exclusivement avec le backend via des appels HTTP (fetch/Axios). Le backend expose une API REST qui gere toute la logique metier, l'accès aux données, et l'authentification. La base PostgreSQL stocke toutes les données de maniere relationnelle.

Structure du projet

Le projet est organise en deux dossiers principaux :

- backend/ : Contient les applications Django (accounts, reclamations, sites_reseau, dashboard, cartographie) et la configuration (config/)
- frontend/ : Contient l'application React avec les pages, composants, API, et styles
- media/ : Stocke les documents generes (PDF, rapports)

2. Backend : Django REST Framework

Le backend est construit avec Django 4.2 et Django REST Framework (DRF). Django est un framework web Python qui suit le pattern Model-View-Template (MVT), mais ici nous utilisons uniquement la partie API (pas de templates HTML, le frontend React gere l'affichage).

Pourquoi Django ?

- ORM puissant pour les requetes complexes sur PostgreSQL
- Admin panel automatique pour la gestion des donnees
- Authentification integree et systeme de permissions
- Ecosystème riche de packages (DRF, Simple JWT, CORS headers)

Structure des applications Django

Chaque module metier est une app Django independante :

[accounts/](#)

Gestion des utilisateurs, authentification JWT, controle d'accès par roles. Contient le CustomUser model, les serializers et les vues pour CRUD.

[reclamations/](#)

Système de tickets de reclation. Gere le cycle de vie des tickets (ouvert/resolu/ferme), l'integration IA pour les descriptions, et les commentaires.

[sites_reseau/](#)

Modele et API pour les sites BTS. Supporte le CRUD, le filtrage par statut, et le soft-delete (archivage).

[dashboard/](#)

Vues analytiques et KPIs. Aggregate les donnees des reclamations et sites pour les graphiques et rapports. Inclut le service de generation IA.

[cartographie/](#)

Endpoint optimise pour les donnees de carte (coordonnees, statut).

[config/](#)

Configuration globale : settings.py (DB, JWT, CORS), urls.py (routage API), wsgi.py.

DRF Serializers

Les serializers DRF convertissent les objets Django en JSON pour l'API et inversement. Ils gerent la validation des donnees d'entree, les relations imbriquees (ex: un ticket contient les infos du site et de l'utilisateur), et les champs ecriture/separation lecture.

DRF Views

Les vues sont des fonctions decorées avec `@api_view` qui gerent les requetes HTTP. Elles utilisent les permissions (`IsAuthenticated`, `AllowAny`) pour controler l'accès. Le pattern est simple : recevoir la requete, valider les donnees, executer la logique, retourner une reponse JSON.

3. Base de Donnees : PostgreSQL

PostgreSQL est utilise comme systeme de gestion de base de donnees relationnelle (SGBDR). C'est un choix solide pour un projet de cette envergure grâce à sa fiabilité, ses performances, et son support avancé des requetes complexes.

Pourquoi PostgreSQL ?

- Support complet du SQL avec des fonctionnalites avancees (CTE, Window Functions)
- Gestion fiable des transactions et de l'integration
- Performance avec les gros volumes de donnees
- Extensions disponibles (PostGIS pour la geolocalisation)

Modeles principaux

La base contient 4 tables principales :

CustomUser

Utilisateurs avec roles (RBAC). Champs : code_user (U001...), email (login), role, first_name, last_name, is_active.

SiteReseau

Sites BTS avec localisation GPS, statut (UP/DOWN), technologie (4G/5G), rayon de couverture. Code auto-generé (S001...).

Reclamation

Tickets de reclamtion lies a un client et un site. Cycle de vie : ouvert -> resolu -> ferme. Integre l'IA pour la description.

RapportIA

Rapports IA sauvegardes avec le prompt, le contenu HTML genere, et les dates de la periode analysee.

Relations entre tables

Reclamation.site -> SiteReseau (FK, SET_NULL si supprime)

Reclamation.cree_par -> CustomUser (FK, agent Call Center)

Reclamation.assigne_a -> CustomUser (FK, ingénieur reseau)

RapportIA.cree_par -> CustomUser (FK, superviseur)

CommentaireTicket.reclamation -> Reclamation (FK, CASCADE)

CommentaireTicket.auteur -> CustomUser (FK, ingénieur)

4. Frontend : React.js

Le frontend est une application React (Create React App) qui communique avec le backend via des appels HTTP. C'est une Single Page Application (SPA) : une seule page HTML est chargée, et React gère la navigation et l'affichage dynamiquement.

Pourquoi React ?

- Composants réutilisables pour un code maintenable
- State management simple avec useState/useEffect
- Large écosystème de bibliothèques (Recharts, Leaflet, React Router)
- Rendu performant grâce au Virtual DOM

Structure du frontend

pages/

6 pages principales : Login, AdminDashboard, CallCenter, EngineerDashboard, SupervisorDashboard, Profile. Chaque dashboard est adapté au rôle.

components/

Composants partagés : DetailModal (vues détaillées avec graphiques), Map (carte Leaflet interactive).

api/

Couche d'accès aux données : axios.js (instance Axios configurée), auth.js (login), dashboard.js (KPIs, rapports), tickets.js (CRUD tickets/sites).

hooks/

Hooks personnalisés : useCountUp (compteur animé), useRipple (effet de clic), spawnParticles (particules).

styles/

CSS global : themes.css (variables CSS, animations).

Routing (React Router)

La navigation est gérée par React Router v6. Chaque rôle a sa propre route : /admin-dashboard, /call-center-dashboard, /engineer-dashboard, /supervisor-dashboard. La racine (/) redirige vers /login. La redirection post-connexion est automatique selon le rôle extrait du token JWT.

State Management

Pas de Redux ni de Context API global. Chaque page gère son propre state avec les hooks React (useState, useEffect). Les données sont fetchées au montage du composant et mises en cache dans le state local. Le token JWT est stocké dans localStorage.

5. Intelligence Artificielle : Google Gemini

L'intégration IA est un composant clé de cette plateforme. Nous utilisons l'API Google Gemini (model gemini-2.5-flash) pour deux fonctionnalités principales :

5.1 Generation automatique de descriptions d'incidents

Quand un agent du Call Center crée un ticket, il saisit des mots-clés rapides décrivant le problème du client. Ces mots-clés sont envoyés à Gemini avec un prompt structure qui demande de les transformer en un rapport d'incident professionnel et structuré.

Le prompt inclut les informations client (nom, téléphone), les mots-clés de l'agent, et des consignes de rédaction (français professionnel, structuration en sections, pas d'invention d'informations techniques). Gemini retourne un texte formaté qui est stocké directement dans le champ description du ticket.

Avantage : l'agent passe 30 secondes à saisir des mots-clés au lieu de 5 minutes à rédiger un rapport complet. L'IA fait le travail de rédaction à sa place.

5.2 Generation de rapports analytiques

Le superviseur peut demander à l'IA de générer un rapport d'analyse réseau. Le système :

1. Collecte les données réelles du réseau (stats sites, tickets, délais)
2. Construit un prompt avec les données + la demande du superviseur
3. Envoie le tout à Gemini avec le rôle d'analyste réseau Djezzy
4. Récupère le HTML généré (tableaux, graphiques, recommandations)
5. L'affiche dans l'interface pour consultation ou export PDF

Le prompt est soigneusement conçu pour que l'IA génère du HTML inline avec du CSS pour un rendu propre directement dans le navigateur, sans balises <html> ou <head>.

6. Authentification : JWT

L'authentification utilise JSON Web Tokens (JWT) via la librairie django-simple-jwt. C'est un standard moderne pour les applications SPA qui remplace les sessions classiques.

Fonctionnement

1. L'utilisateur envoie email + mot de passe a `/api/token/`
2. Le backend verifie les credentials et genere deux tokens :
 - Access token (1h de valide) : contient le role, le code_user
 - Refresh token (7 jours) : permet de regenerer l'access token
3. Le frontend stocke les tokens dans localStorage
4. Chaque requete API inclut le header Authorization: Bearer <token>
5. Le backend verifie la signature du token avant chaque requete

Refresh automatique

Le frontend verifie l'expiration du token a chaque requete. Si le token est expire mais que le refresh token est encore valide, il envoie automatiquement une requete a `/api/token/refresh/` pour obtenir un nouveau access token, sans déconnexion de l'utilisateur.

7. Cartographie : Leaflet.js

La carte interactive utilise Leaflet.js, une bibliotheque JavaScript open source pour les cartes interactives. Elle utilise les tuiles OpenStreetMap pour le rendu de la carte.

Fonctionnalites de la carte

- Marqueurs colores par statut (vert = UP, rouge = DOWN)
- Auto-zoom sur tous les sites au chargement (fitBounds)
- Cercles de couverture au survol (rayon configurable par site)
- Popups avec details du site (nom, code, wilaya, commune, statut)
- Responsive : s'adapte a la taille du conteneur

Donnees affichees

La carte recoit les donnees depuis l'endpoint `/api/dashboard/carte-sites/` qui retourne les coordonnees GPS, le statut, et le nombre de tickets ouverts pour chaque site. Le composant Map.jsx filtre les sites sans coordonnees et rend un Marker + Popup pour chacun.

8. Bibliotheques Supplementaires

Backend

[djangorestframework](#)

Framework API REST pour Django. Fournit les serializers, vues, permissions et la documentation auto de l'API.

[django-simple-jwt](#)

Implementation JWT pour DRF. Gere la generation, validation et refresh des tokens.

[django-cors-headers](#)

Gestion des en-tetes CORS pour autoriser les requetes du frontend React vers le backend Django.

[python-decouple](#)

Lecture des variables d'environnement depuis le fichier .env (cles API, mots de passe DB).

[google-genai](#)

Client Python officiel pour l'API Google Gemini. Utilise pour les fonctionnalites IA.

Frontend

[react-router-dom](#)

Routing cote client pour les SPA React. Gere la navigation entre les dashboards sans rechargement de page.

[axios](#)

Client HTTP pour JavaScript. Utilise pour l'instance API configuree avec intercepteurs JWT.

[recharts](#)

Bibliotheque de graphiques React. Utilise pour les courbes d'evolution, donuts, barres et autres visualisations.

[react-leaflet + leaflet](#)

Composants React pour cartes interactives. Wrapper React autour de Leaflet.js.

[@mui/material \(pas utilise\)](#)

Note : le projet utilise du CSS inline et des styles JavaScript plutot que Material UI.

9. Diagramme des Echanges

Voici le flux de donnees pour les principales operations :

Connexion

React --(POST email+password)--> Django --(verifie DB)--> PostgreSQL

Django --(retourne access+refresh tokens)--> React --(stocke dans localStorage)

Creation de ticket

React --(POST ticket data)--> Django --(sauvegarde en DB)--> PostgreSQL

Django --(appelle Gemini API avec mots-cles)--> Google Gemini

Gemini --(retourne description)--> Django --(met a jour le ticket)--> PostgreSQL

Consultation dashboard

React --(GET /dashboard/stats/)--> Django --(requetes aggregatees)--> PostgreSQL

Django --(retourne JSON avec KPIs)--> React --(affiche graphiques Recharts)

Generation rapport IA

React --(POST prompt)--> Django --(collecte donnees reseau)--> PostgreSQL

Django --(envoie donnees+prompt a Gemini)--> Google Gemini

Gemini --(retourne HTML)--> Django --(retourne a React)--> Affichage dans l'interface

10. Choix Techniques et Justifications

Pourquoi pas de GraphQL ?

Pour un PFE avec 5 endpoints principaux, REST est plus que suffisant. GraphQL ajouterait une complexite inutile (schema, resolvers, complexity analysis) sans benefice significatif pour notre volume de donnees.

Pourquoi pas de Redux ?

Chaque page est relativement independante. Le state local avec useState/useEffect est suffisant et plus simple a maintenir. Si le projet grandissait, on pourrait migrer vers Redux Toolkit ou Zustand.

Pourquoi fetch() + Axios dans le meme projet ?

Le module axios.js est configure avec des intercepteurs pour le refresh automatique. Les autres modules (dashboard.js, tickets.js) utilisent fetch() natif avec leur propre logique de refresh. C'est un choix pragmatique - les deux marchent, l'important est la logique de refresh qui est identique.

Pourquoi Gemini et pas un autre LLM ?

Gemini 2.5 Flash est rapide, peu cher, et bien adapte a la generation de texte en francais. L'API est simple a integrer avec le SDK Python officiel. Le mode flash est parfait pour des taches de generation de texte courtes comme les descriptions d'incidents.

Pourquoi PostgreSQL et pas SQLite ?

SQLite est limite pour un projet avec des requetes complexes (agregations, jointures, tris). PostgreSQL gere parfaitement les requetes d'analyse du dashboard (COUNT, AVG, GROUP BY sur des millions de lignes potentiels). En production, c'est aussi le choix standard pour la fiabilite et la scalabilite.

Pourquoi des styles inline et pas CSS Modules ?

Les styles inline en JavaScript (objet S = {...}) permettent de garder le style avec le composant, de dynamiser les couleurs dynamiquement (selon le statut), et d'eviter les conflits de noms. C'est un choix courant dans les projets React sans architecture CSS enterprise.